

UNIVERSITY OF UP-FAMNIT

BOOKSWAP: DESIGN AND IMPLEMENTATION OF A PEER-TO-PEER BOOK EXCHANGE PLATFORM

Seminar Report

Author: Vanja Antonovic (ID: 89211069)

Course: Systems III (2024/2025)

Abstract

BookSwap is a web-based information system that enables users to discover, offer, and exchange printed books in a trustworthy, convenient, and environmentally responsible way. The platform delivers user registration and authentication, management of book listings with images and metadata, genre categorisation, search and filtering, buyer–seller messaging, and profile management. The current physical database consists of five tables (login, posts, genres, chats, messages) as implemented in MySQL. This report formalises the problem context, elicits functional and non-functional requirements, and presents a corrected logical design that resolves many-to-many relations and introduces auxiliary entities such as Wishlist, Rating, and Order to improve expressiveness and data integrity. The document also outlines the system architecture, implementation overview, testing approach, and future work. Diagrams required by the seminar template are included as placeholders so they can be inserted in final form.

Contents

Abstract	2
1. Introduction.....	5
1.1 Objectives and contributions.....	5
1.2 Stakeholders	5
2. Theoretical background	5
3. System requirements.....	6
3.1 Functional requirements.....	6
3.2 Non-functional requirements	6
4. Feasibility study	6
4.1 Technical feasibility.....	6
4.2 Legal and compliance feasibility	7
4.3 Economic and organisational feasibility	7
5. Logical design.....	7
5.1 User role / function matrix	7
5.2 Data dictionary - current physical database	7
5.3 Data dictionary - corrected logical model (proposed)	9
6. Data modelling artefacts	12
6.1 Entity-Relationship Diagram	12
6.2 Relational model (3NF)	13
6.3 Normalisation notes	13
7. Architecture and object-oriented design	14
7.1 System architecture	14
7.2 UML class diagram.....	14
7.3 UML sequence diagram.....	15
8. Physical model.....	17
9. Implementation	17
10. Testing.....	18
11. Future work.....	18
12. Conclusion	18
References	19

1. Introduction

Despite the rise of e-reading, printed books remain culturally and practically important. Many readers hold books they no longer need, while others seek affordable access to diverse titles. General-purpose marketplaces are not optimised for peer exchange, discovery by genre, or low-friction communication. BookSwap addresses this gap by providing a focused platform where users can list books, search and filter by meaningful attributes, and coordinate exchanges or sales directly with other users.

This seminar follows the official Systems III template. It states the problem, derives requirements, evaluates feasibility, and develops the logical and physical designs. Object-oriented and process-oriented views complement the data model. The report concludes with implementation notes, testing, and future improvements.

1.1 Objectives and contributions

The objectives are to design a robust yet practical information system for book exchange and to document the analysis and design artefacts to the standard required by the course. Contributions include: a clear problem framing; a refined logical schema beyond the initial five tables; consistent diagram notation as required by the template; and an implementation-oriented discussion that maps design decisions to real components.

1.2 Stakeholders

Primary stakeholders are registered users who list or acquire books. Secondary stakeholders include moderators or administrators who maintain catalogues (for example, genres) and ensure acceptable use. External stakeholders include potential partners such as payment providers, if monetisation features are introduced in future work.

2. Theoretical background

The platform is a typical web information system with a browser-based client, a stateless HTTP API, and a relational database. Core design principles include separation of concerns between presentation, application logic, and persistence, and the use of normalisation to reduce redundancy and anomalies. The data model follows standard entity-relationship techniques and is refined into a relational schema in third normal form where feasible. Security principles such as password hashing, least privilege, and input validation are considered foundational.

3. System requirements

3.1 Functional requirements

The system supports the following user-facing capabilities:

User registration and login with profile management.

Create, edit, and delete book listings with title, author, genre, price, description, and image.

Search and filter listings by title, author, genre, condition, and price; sort by recency or price.

Messaging between interested buyers and sellers within the context of a listing.

Notification of unread messages within the interface.

Genre management and browsing by genre.

Responsive design suitable for desktop and mobile use.

3.2 Non-functional requirements

Security: hashed passwords, transport encryption in deployment, server-side validation.

Performance: acceptable response times under typical student-scale usage; indexed lookups on key fields.

Usability and accessibility: clear navigation, readable typography, and sufficient contrast.

Reliability and availability: graceful error handling and routine backups.

Maintainability and scalability: modular code, documented API endpoints, and relational indexes.

4. Feasibility study

4.1 Technical feasibility

The web stack (React, Node.js/Express, MySQL) is mature and appropriate for rapid development. The current physical schema comprises five tables as implemented in MySQL (login, posts, genres, chats, messages). Client-server communication uses HTTP with JSON payloads. Images are stored as file paths in the database.

4.2 Legal and compliance feasibility

User data should be processed according to GDPR principles, with explicit consent and minimised collection. Passwords must be hashed using a strong one-way function. Content policies and reporting features reduce misuse.

4.3 Economic and organisational feasibility

The platform can be hosted on commodity infrastructure. Initial operation requires modest resources and can be maintained by a small team. Monetisation is optional and can be explored later.

5. Logical design

5.1 User role / function matrix

Function	Guest	Registered User	Administrator
Browse and view listings	Yes	Yes	Yes
Search and filter	Yes	Yes	Yes
Create and manage own listings	No	Yes	Yes
Messaging (buyer–seller)	No	Yes	Yes
Manage genres	No	No	Yes
Moderate users and content	No	No	Yes
Wishlist	No	Yes	Yes
Leave ratings	No	Yes	Yes

5.2 Data dictionary - current physical database

The following tables are implemented in the current MySQL database:

Table: login

Attribute	Type	Key/Constraint	Description
id	INT(10)	Primary key	User identifier

name	VARCHAR(255)		Full name
email	VARCHAR(255)	Unique	User email address
password	VARCHAR(255)		Password hash
profile_pic	VARCHAR(255)		Path to profile image

Table: posts

Attribute	Type	Key/Constraint	Description
id	INT(10)	Primary key	Post identifier
title	VARCHAR(255)		Book title
description	TEXT		Listing description
email	CHAR(50)	Foreign key to login.email	Owner email
price	DECIMAL(10,2)		Listed price
picture	VARCHAR(255)		Path to image
time	TIMESTAMP		Created timestamp
author	VARCHAR(255)		Book author
genre	VARCHAR(255)		Genre name or code

Table: genres

Attribute	Type	Key/Constraint	Description
id	INT(10)	Primary key	Genre identifier
name	VARCHAR(255)	Unique (recommended)	Genre name

Table: chats

Attribute	Type	Key/Constraint	Description
id	INT(10)	Primary key	Chat identifier
buyer_id	INT(10)	Foreign key to login.id	Buyer user id
seller_id	INT(10)	Foreign key to login.id	Seller user id
post_id	INT(10)	Foreign key to posts.id	Related post id
postImage	VARCHAR(255)		Reference image path
postTitle	VARCHAR(255)		Reference title

Table: messages

Attribute	Type	Key/Constraint	Description
id	INT(10)	Primary key	Message identifier
chat_id	INT(10)	Foreign key to chats.id	Related chat id
user_email	CHAR(50)	Foreign key to login.email	Author email
message	TEXT		Message body
timestamp	TIMESTAMP		Created timestamp
postImage	VARCHAR(255)		Optional image path
postTitle	VARCHAR(255)		Optional title copy

5.3 Data dictionary - corrected logical model (proposed)

To improve integrity and expressiveness, the corrected logical model replaces email-based foreign keys with integer user identifiers and introduces additional entities. This section documents the proposed entities.

Entity: User

Attribute	Type	Key/Constraint	Description
id	INT	Primary key	User identifier
name	VARCHAR(255)		Full name
email	VARCHAR(255)	Unique	Email address
password_hash	VARCHAR(255)		Hashed password
profile_pic	VARCHAR(255)		Path to profile image
created_at	TIMESTAMP		Creation timestamp

Entity: Book

Attribute	Type	Key/Constraint	Description
id	INT	Primary key	Book identifier
title	VARCHAR(255)		Title
author	VARCHAR(255)		Author
isbn	VARCHAR(20)	Nullable	ISBN if known
description	TEXT		Description
picture	VARCHAR(255)		Image path
condition	ENUM		Book condition

Entity: Genre

Attribute	Type	Key/Constraint	Description
id	INT	Primary key	Genre identifier
name	VARCHAR(255)	Unique	Genre name

Entity: BookGenre (junction)

Attribute	Type	Key/Constraint	Description
book_id	INT	Primary/Foreign key to Book.id	Book id
genre_id	INT	Primary/Foreign key to Genre.id	Genre id

Entity: Listing

Attribute	Type	Key/Constraint	Description
id	INT	Primary key	Listing identifier
seller_id	INT	Foreign key to User.id	Owner user id
book_id	INT	Foreign key to Book.id	Book id
price	DECIMAL(10,2)		Price at offer
status	ENUM		For example: active, reserved, sold
created_at	TIMESTAMP		Creation timestamp

Entity: Chat

Attribute	Type	Key/Constraint	Description
id	INT	Primary key	Chat identifier
listing_id	INT	Foreign key to Listing.id	Listing id
buyer_id	INT	Foreign key to User.id	Buyer user id
seller_id	INT	Foreign key to User.id	Seller user id
created_at	TIMESTAMP		Creation timestamp

Entity: Message

Attribute	Type	Key/Constraint	Description
id	INT	Primary key	Message identifier
chat_id	INT	Foreign key to Chat.id	Chat id
sender_id	INT	Foreign key to User.id	Author user id
body	TEXT		Message body
created_at	TIMESTAMP		Creation timestamp

Entity: Wishlist (junction)

Attribute	Type	Key/Constraint	Description
user_id	INT	Primary/Foreign key to User.id	User id
listing_id	INT	Primary/Foreign key to Listing.id	Listing id
created_at	TIMESTAMP		Added timestamp

Entity: Rating

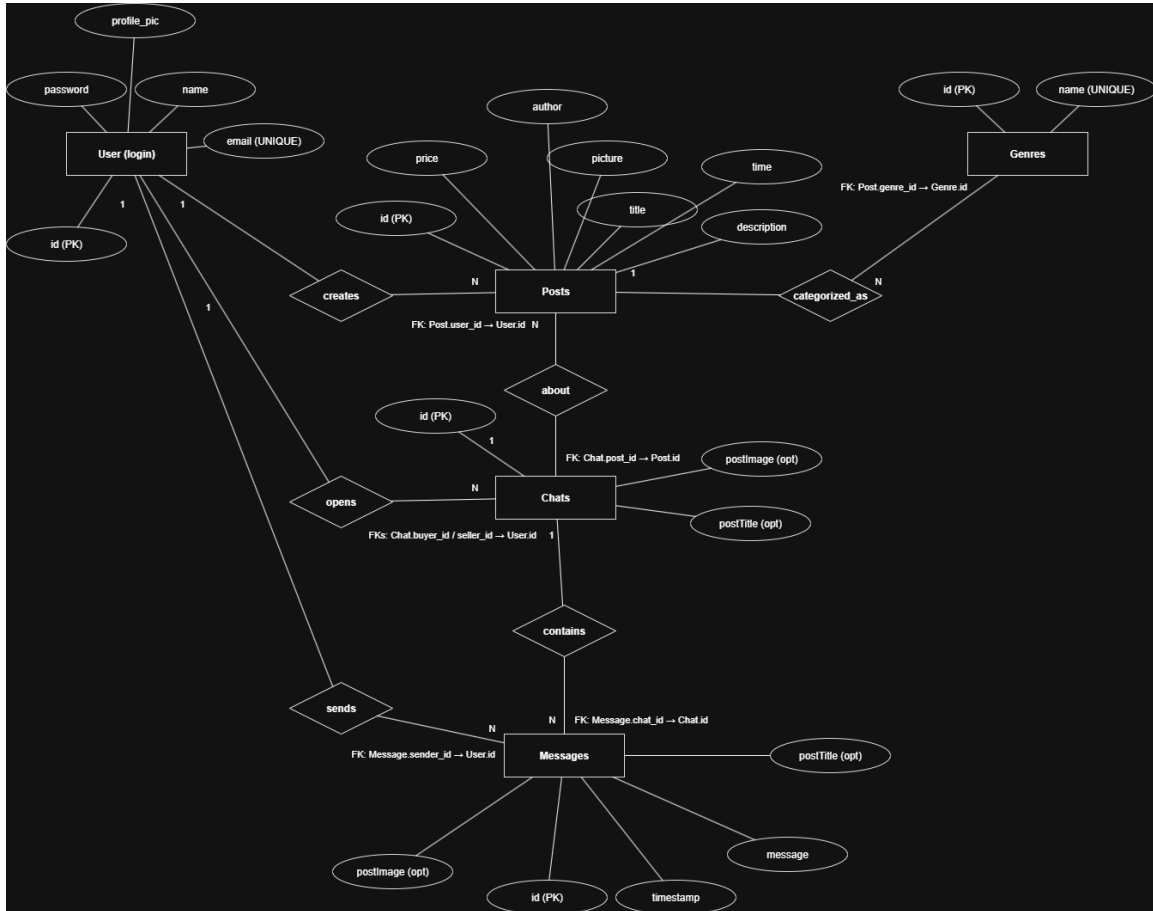
Attribute	Type	Key/Constraint	Description
id	INT	Primary key	Rating identifier
rater_id	INT	Foreign key to User.id	Reviewer user id
ratee_id	INT	Foreign key to User.id	Reviewed user id
score	TINYINT	Check 1..5	Score
comment	TEXT	Nullable	Optional comment
created_at	TIMESTAMP		Creation timestamp

Entity: Order

Attribute	Type	Key/Constraint	Description
id	INT	Primary key	Order identifier
listing_id	INT	Foreign key to Listing.id (unique)	Listing sold
buyer_id	INT	Foreign key to User.id	Buyer user id
price_at_purchase	DECIMAL(10,2)		Captured price
created_at	TIMESTAMP		Creation timestamp

6. Data modelling artefacts

6.1 Entity-Relationship Diagram

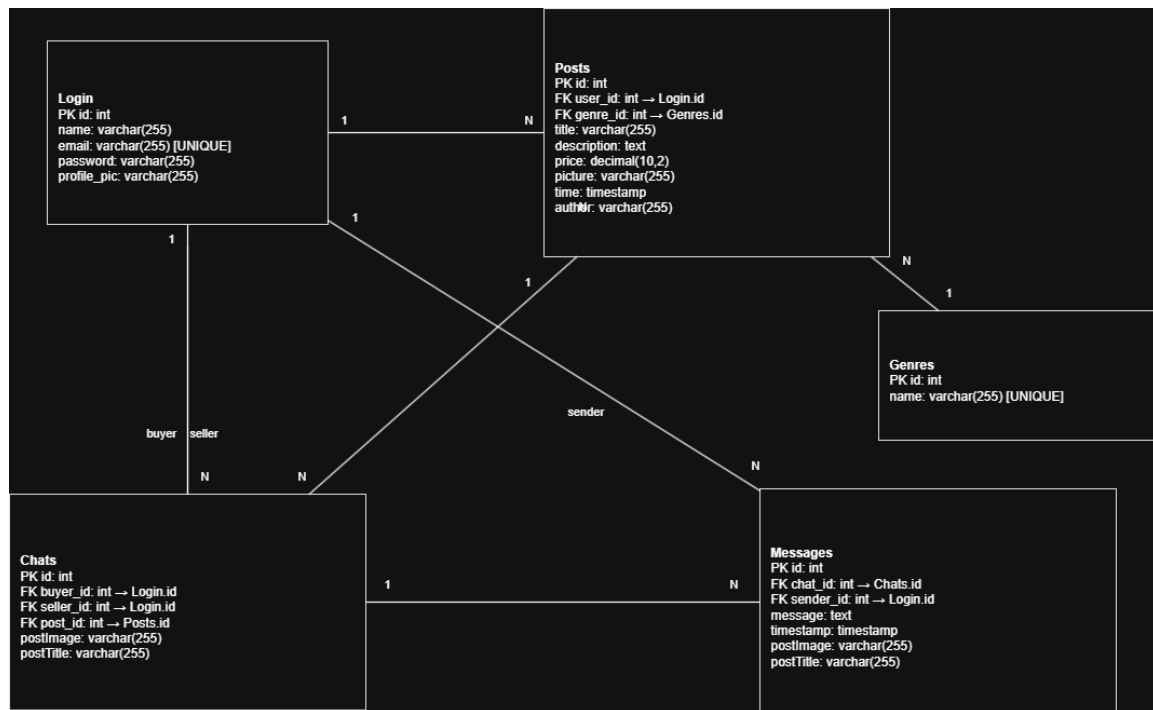


[Figure 6.1: ER diagram.]

6.2 Relational model (3NF)

The relational schema below is consistent with the corrected ER model. For clarity, foreign keys are written in text rather than arrows.

Table	Columns and constraints
Login	id (PK), name, email, password, profile_pic
Posts	id (PK), title, description, email, price, picture, time, author, genre
Genre	id (PK), name
Chats	id (PK), buyer_id (FK to Login.id), seller_id (FK to Login.id), post_id (FK to Posts.id), postImage, postTitle
Messages	id (PK), chat_id (FK to Chats.id), user_email, message, timestamp, postImage, postTitle



[Figure 6.3: Relational model diagram.]

6.3 Normalisation notes

The proposed schema eliminates repeating groups and ensures atomic attributes (first normal form). There are no partial dependencies on part of a composite key (second normal form), and non-key attributes depend only on the key, the whole key, and nothing but the key (third normal form). Junction tables resolve many-to-many relations.

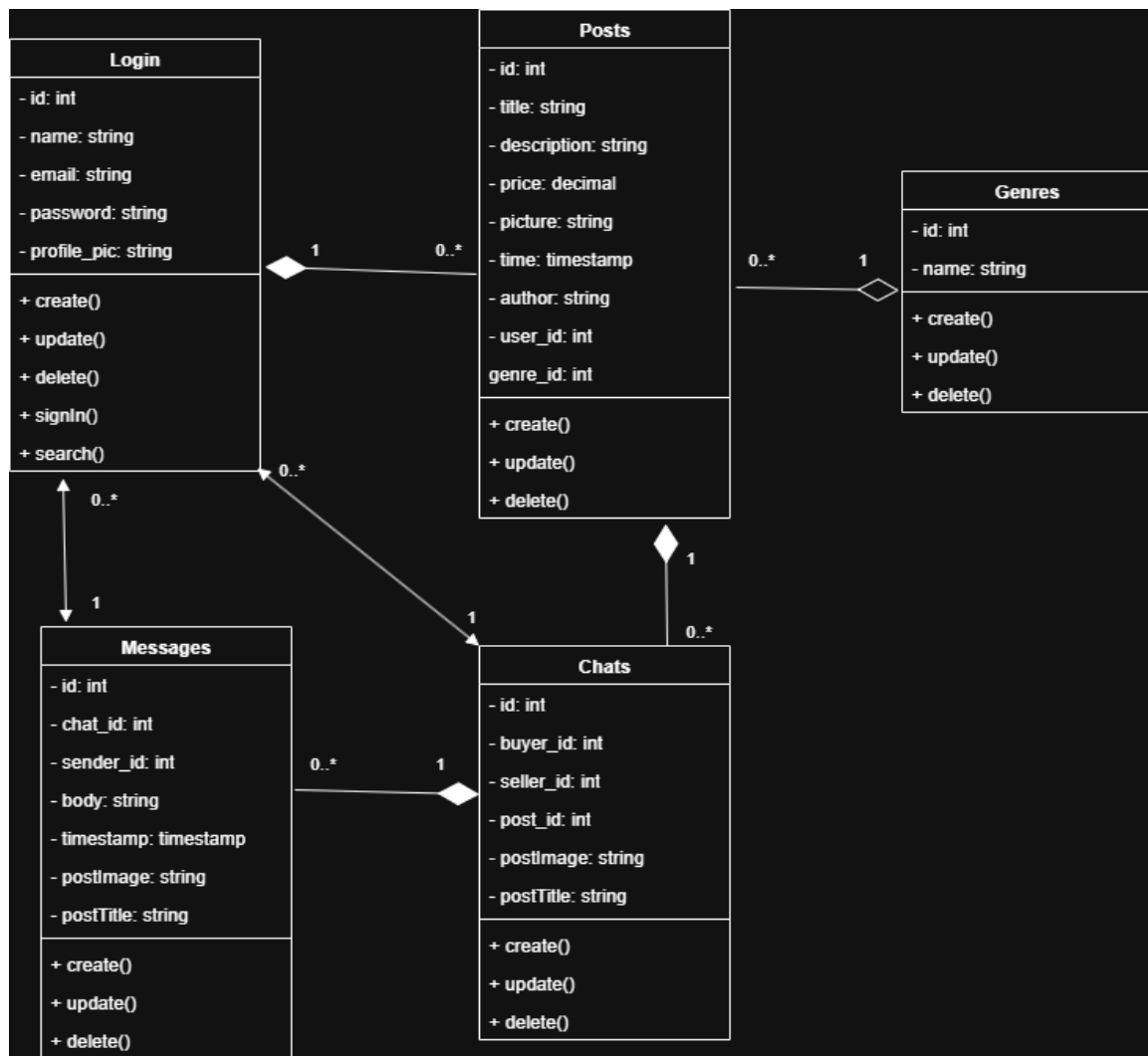
7. Architecture and object-oriented design

7.1 System architecture

Presentation layer: React application with pages and components such as Home, Profile, PostDetail, and OwnPostDetail. Application layer: Node.js/Express services exposing REST endpoints for posts, users, genres, chats, and messages. Data layer: MySQL database with relational constraints and indexes. Static assets (images) are referenced via file paths.

7.2 UML class diagram

The class diagram represents key domain classes such as Login, Chats, Messages, Genres, Posts, including attributes and associations. It supports the same semantics as the data model without implying persistence details.



[Figure 8.2: UML class diagram.]

7.3 UML sequence diagram

This sequence diagram illustrates two separate interaction flows:

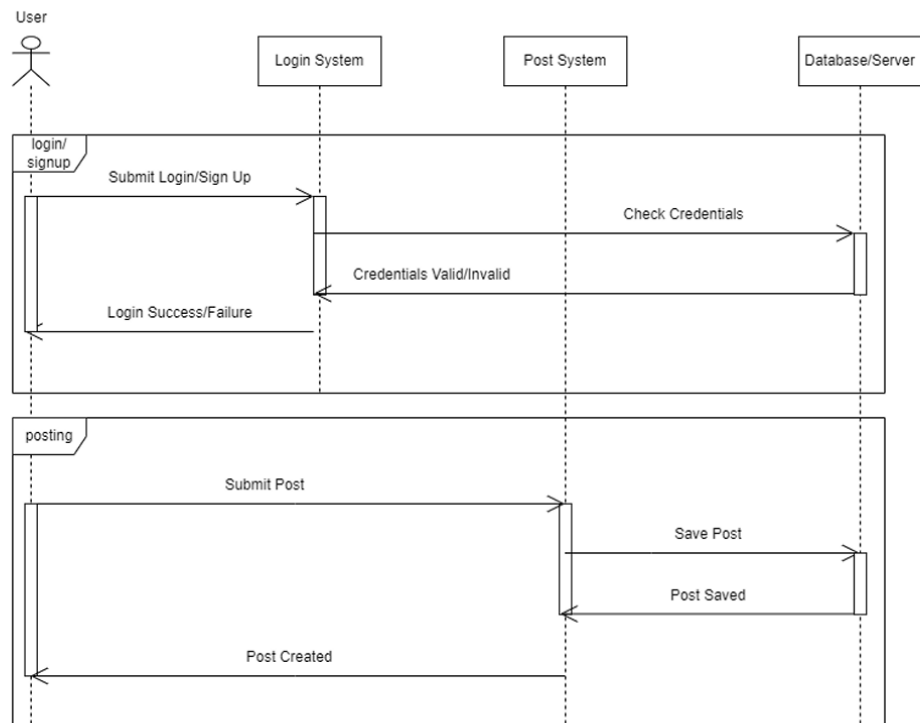
1. Login/Sign Up

- The **User** initiates the process by submitting their login or sign-up credentials to the **Login System**.
- The **Login System** forwards the credentials to the **Post System**, which then requests validation from the **Database/Server**.
- The **Database/Server** checks the credentials and returns the result to the **Post System**, which relays it back to the **Login System**.
- The **Login System** informs the **User** whether the login was successful or failed.

2. Posting

- The **User** submits a new post to the **Post System**.
- The **Post System** saves the post to the **Database/Server**.
- Once saved, the **Database/Server** confirms back to the **Post System**, which then notifies the **User** that the post has been created.

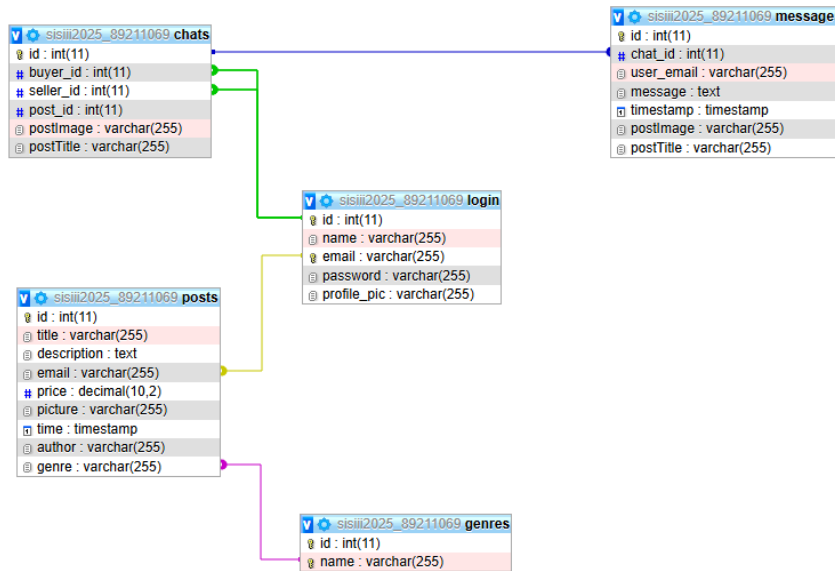
The diagram uses vertical lifelines to represent each participant and horizontal arrows to represent synchronous message exchanges. The shaded activation bars along each lifeline indicate when that participant is active in the interaction. Each flow is grouped into a separate fragment for clarity.



[Insert Figure 8.3a: UML sequence diagram - Contact Seller.]

8. Physical model

The physical model is a faithful implementation of the relational design in MySQL. It defines data types, indexes, and default values. Screenshots from phpMyAdmin Designer should be inserted here as evidence of the implemented schema.



[Figure 9.1: Physical database model (phpMyAdmin Designer).]

Indexing recommendations: B-tree indexes on posts.title, posts.author, posts.genre, and foreign keys used in joins; composite indexes for frequent query predicates where justified.

9. Implementation

The frontend is implemented in React. The Home page retrieves listings with optional search and filter parameters; Profile shows the user's own listings; PostDetail displays a listing and offers actions such as contacting the seller; OwnPostDetail allows the seller to edit or delete their own listing.

Representative HTTP endpoints observed in the current code base include retrieval of all posts with query parameters, retrieval of a single post by identifier, and deletion of a post by identifier. Authentication and messaging endpoints follow the same design pattern and can be extended as needed.

Endpoint (method)	Purpose	Notes
GET /posts	List posts with optional search and filter query parameters	Used by Home page
GET /post/{id}	Fetch a single post by identifier	Used by PostDetail and OwnPostDetail
POST /delete-post	Delete a post by identifier	Used by OwnPostDetail

Frontend routing: navigation to /post/{id} for general view and /own-post/{id} when the current user is the owner of the listing.

10. Testing

Testing activities covered functional scenarios, constraints, and edge cases:

Test case	Preconditions	Steps	Expected result
User registration and login	Empty database or new account	Register, then login with the same email	Session established; profile visible
Create listing	Authenticated user	Fill form, upload image, submit	Listing appears on Home and Profile
Messaging	Buyer and seller accounts	Open listing, start chat, send message	Message stored in messages with correct chat_id

11. Future work

Replace email-based foreign keys with user identifiers consistently across tables.

Introduce wishlist and rating features with appropriate database support.

Payment integration and order capture to support direct purchases where permitted.

Content moderation tools and audit logs for administrative users.

Improve image storage using object storage and signed URLs.

12. Conclusion

BookSwap demonstrates a coherent approach to designing and implementing a focused peer-to-peer exchange system. The refined logical model improves integrity and provides room for new features while keeping the current physical schema usable. The artefacts included here align with the seminar template and can be finalised by inserting the corresponding diagrams.

References

- [1] Official MySQL Documentation.
- [2] React, Node.js, and Express official documentation.
- [3] GDPR guidelines for data processing.